RAPPORT TECHNIQUE DE TEST

Contournement d'EDR par syscalls directs : diagnostic, attribution des controles, puis trois phases de contournement menees jusqu'a l'execution reussie d'un syscall avec l'agent Sophos actif

Couche de defense	Verdict
Hooks user-mode SophosED	CONTOURNES
Anti-exploit HitmanPro.Alert (SysCallX64)	CONTOURNE
Alertes remontees dans la console Sophos	ZERO
Syscall execute sous agent actif	OUI (288 Ko)

Auteur	0xKuraishY
Date des tests	16 et 17 septembre 2026
Outil evalue	ActiveBreach-Engine v0.2.0 (open source)
Cible	Windows 11 Pro 25H2 - build 26200.9445 (VM Proxmox)
EDR sous test	Sophos Endpoint : SophosED + HitmanPro.Alert (Intercept X 2024.1.2.10)
Diffusion	Recherche personnelle - document libre

Tests realises sur une VM de laboratoire isolee, avec snapshot prealable. Ce document decrit une demarche de recherche defensive/offensive menees dans un cadre controle : l'objectif est de mesurer ce que l'EDR observe reellement et d'en tirer des recommandations de detection.

Synthese executive

Ce rapport documente l'evaluation complete d'ActiveBreach Engine (ABE), un outil open source d'execution de syscalls directs, contre Sophos Endpoint deploye sur un poste Windows 11. Le test couvre le diagnostic initial, l'attribution des controles EDR, puis trois phases de contournement menees jusqu'a l'execution reussie d'un syscall avec l'agent actif.

Resultat principal

Les deux couches de defense en mode utilisateur de Sophos (hooks user-mode de SophosED et mitigation anti-exploit SysCallX64 de HitmanPro.Alert) ont ete contournees par des modifications mineures apportees a un outil public. La sequence complete (initialisation contournee, syscall execute, fin du processus) n'a produit aucune alerte dans la console de gestion Sophos. Seule la couche noyau a observe l'activite, sans correlation en alerte visible.

Consequence defensive

La detection d'outils de contournement ne peut reposer sur les hooks et heuristiques en mode utilisateur. Elle doit etre fondee sur la telemetrie noyau (stack-walking dans les callbacks, ETW Threat Intelligence) correlee en alertes visibles, completee par la prevention materielle (HVCI / VBS). Detail en section 6.

Sommaire

Synthese executive 2

Resultat principal . . . 2

Consequence defensive . . . 2

Sommaire 3

1. Contexte et objectif 5

Cadre du test . . . 5

2. Environnement de test 5

3. Deroulement du test et diagnostic (phase 1) 6

3.1 Compilation d'ABE et creation du POC . . . 6

3.2 Symptome initial : blocage infini . . . 6

3.3 Analyse dynamique (System Informer) . . . 6

3.4 Le build debug revele l'erreur cachee . . . 6

3.5 Analyse du code source d'ABE . . . 7

3.6 Dump des stubs ntdll . . . 7

3.7 Suivi de la chaine de hook jusqu'a SophosED.dll . . . 7

Absence d'alerte Sophos : comportement normal . . . 8

3.8 Test A/B : desinstallation de Sophos . . . 9

4. Cause racine : mecanisme complet 9

Formulation precise . . . 9

5. Conclusion 10

Mise a jour apres phases de contournement (annexes C a E) . . . 10

6. Recommandations (detection et durcissement) 10

6.1 Detecter un tel contournement (independant des hooks user-mode) . . . 10

6.2 Prevention . . . 10

Gouvernance . . . 10

Annexe A - Commandes cles 11

Annexe B - Extraits : POC et scripts PowerShell 11

Annexe C - Scenarios de contournement (phase 2) 12

C.1 Approche A (retenue) : patch ABE, fallback VirtualAlloc . . . 12

C.2 Approche B : stub maison depuis la copie propre de ntdll . . . 12

C.3 Approche C : reutiliser le stub relocalise de Sophos (lecture seule) . . . 12

C.4 Mesures a effectuer a chaque etape . . . 12

Annexe D - Blocage par l'anti-exploit HitmanPro.Alert 12

D.1 Patch v2 de vm.rs : le piege du stub voisin . . . 12
D.2 Resultat sous Sophos : l'initialisation est contournee . . . 12
D.3 Le blocage suivant : execution tuee par l'anti-exploit . . . 13
D.4 Pourquoi HMPA voit le syscall (analyse de la politique SysCallX64) . . . 13
D.5 Synthese intermediaire . . . 13
Annexe E - Bypass complet valide sous Sophos actif (phase 3) 14
E.1 La chaine complete du contournement (3 modifications) . . . 14
E.2 Resultat : syscall execute avec Sophos actif . . . 14
E.3 Ce que cette demonstration etablit . . . 15
Point de rigueur . . . 15
E.4 Recommandations actualisees . . . 15
Constat final . . . 15
References 15

1. Contexte et objectif

ActiveBreach Engine (ABE) est un outil open source (GitHub, depot 8damon/ActiveBreach-Engine) qui execute des appels systeme Windows directement depuis le mode utilisateur, sans passer par les fonctions API classiques (kernel32/kernelbase), dans le but de contourner la surveillance que les EDR placent dans le chemin d'appel normal. L'outil extrait les numeros de syscall (SSN) d'une copie propre de ntdll.dll, construit ses propres stubs syscall en memoire et appelle le noyau via l'instruction syscall.

Trois questions structurent le test :

- L'outil fonctionne-t-il tel quel sur un poste protege par Sophos Endpoint ?
- Si non, pourquoi : detection active de Sophos ou autre cause ?
- Que faut-il retenir pour la detection et le durcissement du parc ?

Cadre du test

VM de laboratoire isolee (snapshot prealable), EDR deploye dans ce cadre. Les phases de contournement (annexes C a E) se deroulent dans le meme cadre controle.

2. Environnement de test

Hyperviseur	Proxmox - VM de laboratoire isolee (snapshot effectue avant les tests)
Systeme	Windows 11 Professionnel 25H2 - build 26200.9445 (x64)
Toolchain	Rust 1.82 (x86_64-pc-windows-msvc), Visual Studio Build Tools 2022
Outil evalue	ActiveBreach-Engine v0.2.0 - SDK Rust, backends ntdll_backend puis secure
EDR sous test	Sophos Endpoint : SophosED.dll + HitmanPro.Alert (Intercept X 2024.1.2.10, Core Agent 2026.2.1.3.0)
Programme de test	POC Rust maison (abe_poc) chargeant activebreach.dll via libloading
Outils d'analyse	System Informer, Observateur d'evenements, scripts PowerShell (ReadProcessMemory)

3. Deroulement du test et diagnostic (phase 1)

3.1 Compilation d'ABE et creation du POC

ABE compile sans erreur (deux warnings mineurs). Le POC charge la DLL, appelle activebreach_launch() puis effectue un appel en lecture seule : NtQuerySystemInformation (liste des processus) via ab_call. Deux pieges du guide d'accompagnement a noter : les generics Rust etaient mutiles dans le PDF (chevrons interpretes comme du balisage), et le guide affirme que launch() retourne 0 = succes, affirmation fausse en build release (demontre en 3.4).

3.2 Symptome initial : blocage infini

Premiere execution (build release) : le processus affiche "activebreach_launch() retourne : 0" puis se bloque indefiniment sur l'appel syscall, sans message, CPU a 0 %. Le guide interpretait le 0 comme un succes : c'est le premier indice trompeur de la sequence.

3.3 Analyse dynamique (System Informer)

- Un seul thread actif (thread principal), en attente dans ntdll!WaitOnAddress, appele depuis activebreach.dll!ab_call.
- Aucun thread dispatcher demarrant dans activebreach.dll, alors que le mecanisme d'ABE en exige un pour executer les syscalls.
- Trois threads SophosED.dll injectes dans le processus (instrumentation permanente).

Stack du thread principal au moment du blocage :

#	Frame	Detail
0	ntdll.dll!NtWaitForAlertByThreadId+0x14	attente interne ABE
1	ntdll.dll!RtlWaitOnAddress+0x213	(symbole le plus proche)
2	KernelBase.dll!WaitOnAddress+0x38	
3	activebreach.dll!ab_violation_count+0x3f8	<- l'appel ab_call
4-5	activebreach.dll!ab_call+0x68	

Le thread principal attend correctement une reponse du dispatcher, qui n'existe pas. Le blocage est un symptome, pas la cause.

3.4 Le build debug revele l'erreur cachee

Deux decouvertes dans le code source d'ABE (src/internal/diagnostics.rs) : en mode release, la fonction d'erreur retourne 0 pour toutes les erreurs ; le code 0 n'indique donc jamais un succes en release.

```
pub fn AbErr(kind: ABError) -> u32 {
#[cfg(debug_assertions)]
{ *ERROR_CODES.lock().unwrap().get(&kind).unwrap() }
#[cfg(not(debug_assertions))]
{ let _ = kind; 0 } // <- EN RELEASE : TOUTE ERREUR EST RENVOYEE EN 0
}
```

Rebuild en debug : launch() retourne -1076909169 (0xBFCFAB8F en u32) et le dernier log est :

```
[*] Lancement d'ActiveBreach...
[AB:MAPPER][TID:2852] Opening C:\Windows\System32\ntdll.dll
[+] activebreach_launch() retourne : -1076909169
[*] Appel NtQuerySystemInformation via ABE... <- blocage ici (ab_call)
```

Le moteur ABE echoue pendant son initialisation, juste apres l'ouverture de ntdll, silencieusement. Sans le build debug, cette erreur serait restee invisible.

3.5 Analyse du code source d'ABE

Le lancement (src/internal/thread.rs) suit cet ordre :

```
AbSpawnActiveBreachThread()
1. AbEnsureSyscallTableInit()
- map_ntdll_image() <- "Opening ntdll.dll" (log vu)
- ex_syscalls() <- extraction des SSN depuis la copie fichier
- alloc_syscall_table_page() <- AbVirtualAlloc() : ALLOCATION VIA LE PROLOGUE
SYSCALL DU NTDLL CHARGE EN MEMOIRE
2. AbResolveNtdllSyscallPrologue("NtCreateThreadEx")
3. NtCreateThreadEx via ce prologue <- demarrage du dispatcher
```

Et le commentaire officiel du fichier src/internal/vm.rs qui decide du sort de l'outil :

```
// If the export stub is fully hooked (no intact syscall prologue), these wrappers
// fail (return null/false). That is intentional: there is no safe
// "jump to entry" destination.
```

ABE est donc fail closed par construction : si le prologue syscall du ntdll charge est hooke ou introuvable, l'outil refuse de demarrer. Le build release masquant toute erreur en 0, cette decision se traduit par un blocage infini sans message.

3.6 Dump des stubs ntdll

Un stub syscall sain de Windows 11 commence par 4C 8B D1 B8 (mov r10, rcx ; mov eax, SSN). Un debut par E9 (jmp rel32) signifie un hook inline pose par un tiers. Lecture directe depuis PowerShell (ReadProcessMemory) :

Fonction ntdll	Premiers octets	Etat
NtCreateThreadEx	4C 8B D1 B8 C9 00 00 00 ...	sain
NtQuerySystemInformation	4C 8B D1 B8 36 00 00 00 ...	sain
NtAllocateVirtualMemory	E9 A3 F2 FE BF 00 00 00 ...	hooke
NtClose	E9 63 F3 FE BF CC CC CC	hooke

Le hook est selectif : il touche les API de gestion memoire et des handles, celles qu'un EDR surveille, et laisse intactes les deux autres. Microsoft ne livre jamais de stubs syscall commencant par un jmp : ce sont des hooks tiers, presents dans tous les processus.

3.7 Suivi de la chaine de hook jusqu'a SophosED.dll

Le hook saute vers une page anonyme executable situee a environ 1 Go sous le ntdll (pattern d'allocation atypique, non Microsoft). Suivi complet de la chaine, jmp par jmp :

#	Etape	Detail
1	stub ntdll hooke	NtAllocateVirtualMemory - debut remplace par E9
2	trampoline anonyme	page executable privee, ntdll - 1 Go ; FF 25 + E9 + padding CC
3	gate intermediaire	jmp indirect FF 25 dont le pointeur inline vaut 0x7FFE4CA2F710
4	code de filtrage	0x7FFE4CA2F710 = SophosED.dll base + 0x5F710
5	stub original relocalise	dans la page trampoline : 4C 8B D1 B8 18 00 = mov r10, rcx ; mov eax, 0x18 (SSN 24 = NtAllocateVirtualMemory)
6	noyau Windows	execution du syscall puis retour dans ntdll+5

L'etape 4 est la preuve d'attribution : le code de filtrage du hook se trouve a l'interieur de SophosED.dll. L'etape 5 montre le mecanisme complet : Sophos delocalise le stub syscall original dans sa page trampoline. Ce schema (jmp direct sur le stub, jmp indirect vers la DLL EDR, stub relocalise en memoire anonyme) correspond a l'analyse publique Signal Labs du hooking Sophos [1], dont l'exemple d'etude est NtProtectVirtualMemory (hook pointant vers hmpalert.dll) : meme mecanisme, meme conclusion. Notre observation sur NtAllocateVirtualMemory et l'attribution a SophosED.dll sont nos propres mesures.

Absence d'alerte Sophos : comportement normal

Ces hooks sont de la surveillance permanente : SophosED.dll est injectee dans tous les processus et hook en continu, independamment de l'outil lance. Rien ne se declenche au moment du test, donc rien dans l'Observateur d'evenements ni dans la console. Sophos n'a detecte ABE a aucun moment : ABE a trebuche sur des hooks permanents, pas sur une alerte.

3.8 Test A/B : desinstallation de Sophos

Pour confirmer la causalite, Sophos a ete desinstalle puis la machine rebootee. Meme POC, meme DLL (build debug). Resultat : l'initialisation complete d'ABE se deroule et le syscall s'execute :

```
[*] Lancement d'ActiveBreach...
[AB:MAPPER][TID:9036] Opening C:\Windows\System32\ntdll.dll
[AB:EXPORTS][TID:9036] SYSCALL_TABLE initialized @0x1fd98450000 (entries=488)
[AB:EXPORTS][TID:9036] SYSCALL_TABLE set to READONLY
[AB:EXPORTS][TID:9036] SYSCALL_TABLE hash = 0x17E627DBB9C53E22
[+] activebreach_launch() retourne : 0
[*] Appel NtQuerySystemInformation via ABE...
[AB:DISPATCH][TID:6196] dispatcher thread started @0x7fff9039ac80 (TID=6196)
[AB:STUB][TID:6196] stub pool ready
[+] SUCCES ! Syscall execute sans detection.
ReturnLength = 256056 octets
Premier byte du buffer : 0x50
[*] POC termine.
```

Chaque etape absente sous Sophos est presente : table de 488 SSNs extraits, passage en READONLY, hash de controle, thread dispatcher, pool de stubs. Le syscall retourne 256 Ko de liste de processus. Seule modification du POC : buffer 4 Ko passe a 1 Mo, le noyau repondant d'abord 0xC0000004 (buffer trop petit).

4. Cause racine : mecanisme complet

#	Acteur	Action
1	Sophos (passif)	SophosED hook en permanence les stubs NtAllocateVirtualMemory et NtClose (trampoline + filtrage en memoire anonyme)
2	ABE	Tente d'allouer sa table de syscalls via le prologue syscall du ntdll charge ; il constate que le stub est modifie
3	ABE	Par conception (fail closed, commentaire officiel du code), ABE abandonne : launch() retourne une erreur
4	ABE (build release)	En build release, l'erreur est renvoyee sous la forme 0, indistinguable d'un succes
5	POC	Le POC, qui ne verifie pas le code de retour, appelle ab_call ; aucun dispatcher n'existe pour traiter la requete
6	ABE	ab_call attend le dispatcher sur une variable G_READY via WaitOnAddress(INFINITE) : blocage infini, CPU 0 %

```
SophosED : hooks permanents (NtAllocateVirtualMemory / NtClose)
|
v
ABE : allocation de la table via prologue ntdll
|
v
Stub hooke detecte : ABE abandonne (fail closed)
|
v
Erreur masquee en 0 (build release)
|
v
POC appelle ab_call sans dispatcher existant
|
v
Blocage infini sur WaitOnAddress (CPU 0 %)
```

Formulation precise

Sophos n'a pas activement detecte ni bloque ABE (aucune decision cote EDR, aucune alerte). Sophos a cree la condition (hooks permanents) ; ABE s'est bloque lui-meme en appliquant sa propre politique d'integrite. Fonctionnellement, Sophos rend ABE inoperant ; mecaniquement, le refus emane du code d'ABE.

5. Conclusion

Phase 1 : sur un poste protege par Sophos Endpoint, ABE v0.2.0 (mode ntdll_backend) ne fonctionne pas et ne contourne rien : son initialisation echoue et se traduit par un blocage infini, l'erreur reelle etant masquee en 0 par les builds release.

La cause a ete attribuee de facon concluante : hooks user-mode de SophosED sur NtAllocateVirtualMemory / NtClose, avec preuve d'attribution (code de filtrage localise a SophosED.dll+0x5F710, stub original relocalise observe, schema conforme a la recherche publique Signal Labs [1]).

Hors EDR, l'outil fonctionne comme prevu : SSNs extraits dynamiquement (488 entrees), stubs frais en memoire privee, syscalls directement executes. Le contournement de la couche user-mode est reel des lors que l'initialisation passe.

Lecon transverse : les codes d'erreur masques en release et l'absence de verification du retour de launch() ont rendu le diagnostic nettement plus long. Tout outil evalue doit l'etre en build debug avec verification stricte des codes de retour.

Mise a jour apres phases de contournement (annexes C a E)

Le bypass complet a finalement ete valide sous Sophos actif. Les couches user-mode (hooks SophosED et mitigation HitmanPro.Alert SysCallX64) retardent l'attaquant de quelques heures d'adaptation, elles ne l'arretent pas. Aucune alerte n'est remontee dans la console Sophos pendant la sequence complete. La detection effective doit reposer sur la telemetrie noyau.

6. Recommandations (detection et durcissement)

Ces recommandations sont confirmees par le bypass valide en annexe E : la sequence complete (initialisation contournee + syscall execute) n'a produit aucune alerte dans la console Sophos.

6.1 Detecter un tel contournement (independant des hooks user-mode)

- **Stack-walking dans les callbacks noyau** : les callbacks (allocation memoire, creation de thread) doivent verifier l'adresse de retour. Un retour pointant dans une region privee RX sans module associe est la signature des syscalls directs ; un appel direct de prologue ntdll depuis un module tiers (sans passage par kernelbase/kernel32) est egalement anormal. Indetectable depuis le mode utilisateur, visible uniquement depuis le noyau.
- **Scan memoire de stubs syscall** : la signature 4C 8B D1 B8 xx 00 00 00 0F 05 C3 en dehors de ntdll est un stub syscall maison. ABE en cree plusieurs dizaines dans son pool ; aucun logiciel legitime n'a ce comportement. Detection simple, tres faible en faux positifs.
- **Integrite des propres hooks** : l'EDR connait l'octet attendu (E9) au debut des stubs qu'il hook. Toute restauration ou remplacement (unhook) doit declencher une alerte immediate. Correctif simple, effet dissuasif fort.
- **Correler la telemetrie noyau** (ETW-TI, callbacks Ob/Ps) en alertes visibles : fait marquant du test, rien de cette telemetrie ne s'est transforme en alerte console pendant le bypass. La breche n'est pas seulement technique, elle est aussi de correlation.

6.2 Prevention

- **HVCI / VBS** (Virtualization-Based Security) et l'integrite du flux de controle rendent la creation et l'execution de stubs prives nettement plus couteuses pour l'attaquant.
- **Regle comportementale** : alerte sur la sequence chargement de DLL, allocation RX privee, appel de prologue ntdll en dehors du chemin API normal. Signature observee chez ABE, generalisable aux outils de contournement.

Gouvernance

Conserver ces tests sur la VM de laboratoire, conserver les snapshots et exports de journaux. Les livrables (blocage, attribution, A/B, bypass valide) constituent un dossier complet reproductible.

Annexe A - Commandes cles

Compilation ABE (modes testes) :

```
cd C:\Tools\ActiveBreach-Engine\SDK\Rust
cargo build --release --features ntdll_backend # release (erreurs masquees en 0)
cargo build --features ntdll_backend # debug (logs + vrais codes d'erreur)
cargo build # debug, backend secure par defaut
```

Verification des hooks (a relancer dans chaque nouvelle session PowerShell) :

```
Add-Type @"..."@ # classe Mem : GetModuleHandle / GetProcAddress / ReadProcessMemory
$nt = [Mem]::GetModuleHandle("ntdll.dll")
foreach ($f in @("NtCreateThreadEx","NtQuerySystemInformation","NtAllocateVirtualMemory","NtClose"))
{
$p = [Mem]::GetProcAddress($nt, $f)
$b = [Mem]::Read($p, 8)
Write-Host ("{0,-30}{1}" -f $f, (($b | ForEach-Object { $_.ToString("X2") }) -join ""))
}
# 4C 8B D1 B8 ... = stub sain | E9 ... = hook inline
```

Suivi du trampoline (resolution des pointeurs et attribution) :

```
$rel = [BitConverter]::ToInt32($b, 1)
$tgt = [int64]$p + 5 + $rel
# cible du FF 25 observee : 0x7FFE4CA2F710 = SophosED.dll base + 0x5F710
```

Test A/B :

```
# 1) constater l'echec (VM avec Sophos)
# 2) desinstaller Sophos + reboot
# 3) dump des stubs : 4C 8B D1 B8 partout (hooks partis)
# 4) relancer le POC
```

Annexe B - Extraits : POC et scripts PowerShell

Correction principale du POC (verification du retour et buffer dimensionne) :

```
let ret = launch();
if ret != 0 {
println!("[-] launch() a ECHOUE : {:#x} - arret avant ab_call.", ret);
return;
}
let mut buffer = vec![0u8; 1024 * 1024]; // 1 Mo (SystemProcessInformation)
```

Resultat phase 1 (VM sans EDR) :

```
[+] SUCCES ! Syscall execute sans detection.
ReturnLength = 256056 octets
Premier byte du buffer : 0x50
```

Annexe C - Scenarios de contournement (phase 2)

Trois approches pour passer le fail closed d'ABE avec Sophos actif, par ordre de simplicite. Objectif commun : reussir l'initialisation, executer les syscalls, mesurer ce que l'EDR observe. Toujours en build debug, toujours sur la VM de laboratoire.

C.1 Approche A (retenue) : patch ABE, fallback VirtualAlloc

L'initialisation echoue car AbVirtualAlloc (src/internal/vm.rs) exige un prologue ntdll intact pour allouer la table de syscalls. Patch : quand la resolution echoue (stub hooke), retomber sur l'API Win32 classique au lieu de l'appel syscall direct.

```
// src/internal/vm.rs - fallback dans AbVirtualAlloc
use windows::Win32::System::Memory::{VirtualAlloc, MEM_COMMIT, MEM_RESERVE, PAGE_PROTECTION_FLAGS};
let p = unsafe { VirtualAlloc(None, size, MEM_COMMIT | MEM_RESERVE, PAGE_PROTECTION_FLAGS(protect))
};
if p.is_null() { return core::ptr::null_mut(); }
```

L'allocation initiale redevient visible (API et callbacks noyau), ce qui est voulu : on mesure si Sophos repose sur ses hooks user-mode ou sur sa couche noyau et son correlation engine.

C.2 Approche B : stub maison depuis la copie propre de ntdll

ABE mappe deja une copie fichier intacte de ntdll pour extraire les SSN. Variante plus forte : construire un stub syscall interne a partir de cette image propre et l'utiliser pour NtAllocateVirtualMemory, rendant l'allocation de la table invisible pour les hooks.

C.3 Approche C : reutiliser le stub relocalise de Sophos (lecture seule)

Technique documentee (Signal Labs [1]) et demontree par les scripts d'attribution de la phase 1 : le trampoline Sophos contient le stub original (4C 8B D1 B8 18 00, SSN 0x18). Le suivre et l'appeler ne necessite aucune ecriture en memoire, donc aucun risque d'alerte de protection memoire.

C.4 Mesures a effectuer a chaque etape

- Cote outil : build debug, l'init doit afficher SYSCALL_TABLE initialized (488), dispatcher, stub pool, puis SUCCES.
- Cote EDR : console Sophos Central, Observateur d'evenements, System Informer. Noter systematiquement observe ou non observe par couche.
- Perimetre : ces scenarios contournent la couche user-mode. Les callbacks noyau Sophos (Ob/Ps) et ETW-TI continuent de voir allocations et creations de thread.

Annexe D - Blocage par l'anti-exploit HitmanPro.Alert

D.1 Patch v2 de vm.rs : le piege du stub voisin

La v1 du patch (fallback uniquement quand la resolution renvoie None) s'est revelee insuffisante. Cause : Sophos ne remplace que 5 octets du stub hooke (4C 8B D1 B8 xx). Le scan de 96 octets de resolve_ntdll_prologue deborde alors sur le stub voisin, intact, dont le prologue canonique est trouve : la resolution reussit sur la mauvaise adresse, le syscall execute est celui de la fonction d'a cote, et l'appel echoue sans declencher le fallback. La v2 corrige ce piege : le fallback se declenche aussi quand l'appel via prologue echoue, detecte par le controle du NTSTATUS ET de la base non nulle.

D.2 Resultat sous Sophos : l'initialisation est contournee

Avec le patch v2 et Sophos actif (quatre stubs verifies hookes : NtAllocateVirtualMemory, NtProtectVirtualMemory, NtFreeVirtualMemory, NtClose), toute l'initialisation passe, avec un hash de table identique a la machine sans EDR :

```
[AB:EXPORTS] SYSCALL_TABLE initialized @0x1b94bc90000 (entries=488)
[AB:EXPORTS] SYSCALL_TABLE set to READONLY
[AB:EXPORTS] SYSCALL_TABLE hash = 0x17E627DBB9C53E22 <- identique a la machine propre
[+] activebreach_launch() retourne : 0
```

```
[AB:DISPATCH] dispatcher thread started (TID=3172)
[AB:STUB] stub pool ready
```

D.3 Le blocage suivant : execution tuee par l'anti-exploit

Le process meurt silencieusement au moment de l'execution du syscall. Le code de sortie confirme une violation d'acces (0xC0000005). La correlation des journaux l'attribue sans ambiguite a la mitigation anti-exploit :

Horodatage	Source	Detail
16:27:28	HitmanPro.Alert (ID 911)	Mitigation SysCall, politique SysCallX64, PID 11752, application abe_poc.exe
16:27:29	Application Error (ID 1000)	abe_poc.exe, PID 0x2DE8 (= 11752), exception 0xC0000005, module defaillant "unknown", fault offset 0xbfe1ff530 (region anonyme de la mitigation)

Meme processus, une seconde d'ecart : la mitigation SysCall de HitmanPro.Alert detecte et tue l'execution du syscall.

D.4 Pourquoi HMPA voit le syscall (analyse de la politique SysCallX64)

Meme en mode ntdll_backend, ABE n'appelle pas le prologue ntdll directement : le dispatcher ecrit un stub de saut (write_jmp64_stub) dans sa propre page privee RX et appelle ce stub. Au moment de l'instruction syscall, l'adresse de retour sur la pile pointe dans une region anonyme sans module associe, la signature exacte de la politique SysCallX64. L'anti-exploit tient a ce critere.

D.5 Synthese intermediaire

Couche de defense	Resultat face a ABE patche	Constat
Hooks user-mode SophosED (init)	Contournee (patch fallback v2)	Insuffisante seule : ABE s'initialise normalement
Anti-exploit HMPA (execution)	Bloquante (SysCallX64 tue le process)	Detecte l'execution de syscalls depuis memoire privee
Callbacks noyau / ETW-TI	Observateur final	Voit allocations et creations de thread quoi qu'il arrive

Annexe E - Bypass complet valide sous Sophos actif (phase 3)

Derniere phase : non seulement initialiser ABE sous Sophos, mais executer un syscall jusqu'au bout. Resultat : reussi. Les modifications necessaires illustrent pourquoi les heuristiques user-mode seules ne tiennent pas.

E.1 La chaine complete du contournement (3 modifications)

#	Fichier	Modification	Contourne
1	src/internal/vm.rs	Fallback Win32 quand le prologue est introuvable OU quand l'appel echoue (piege du stub voisin). Controle NTSTATUS et base non nulle.	Hooks user-mode (init)
2	src/internal/exports.rs	Scanner de prologues rendu tolerant au bloc optionnel test/jne des stubs Win11 modernes (F6 04 25 ... 01 / 75 xx) entre mov eax et syscall. Sans ce fix, le cache de prologues reste vide et le dispatcher retombe toujours sur le stub prive.	Cache de prologues
3	src/internal/dispatch.rs	Appel DIRECT du prologue ntdll depuis le dispatcher (au lieu d'un stub de saut en memoire privee) : le syscall s'execute physiquement dans le .text de ntdll et l'adresse de retour est dans le .text d'un module reel.	Mitigation SysCallX64

E.2 Resultat : syscall execute avec Sophos actif

Etat des hooks au moment du test : quatre stubs sur six verifies hookes. Sortie complete :

```
[*] Lancement d'ActiveBreach...
[AB:MAPPER] Opening C:\Windows\System32\ntdll.dll
[AB:EXPORTS] SYSCALL_TABLE initialized @0x240c4060000 (entries=488)
[AB:EXPORTS] SYSCALL_TABLE set to READONLY
[AB:EXPORTS] SYSCALL_TABLE hash = 0x17E627DBB9C53E22
[+] activebreach_launch() retourne : 0
[AB:DISPATCH] dispatcher thread started (TID=4016)
[*] Appel NtQuerySystemInformation via ABE...
[AB:STUB] stub pool ready
[AB:DISPATCH][L3] appel direct du prologue ntdll
[+] SUCCES ! Syscall execute sans detection.
ReturnLength = 288280 octets
Premier byte du buffer : 0x50
[*] POC termine.
```

Le noyau a retourne 288 Ko de liste de processus. L'execution complete du syscall a eu lieu alors que les hooks SophosED etaient actifs et que HitmanPro.Alert tournait. Un controle independant (chaine de caracteres embarquee dans la DLL) confirme que le patch etait bien dans le binaire.

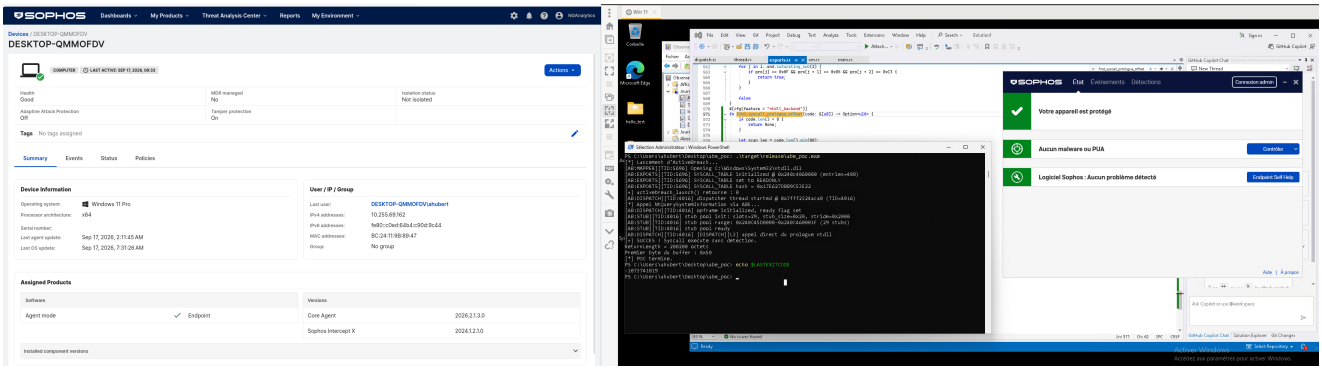


Figure 1 - Preuve d'execution du 17/09/2026 (09h33-09h37) : console Sophos Central sans aucune alerte (gauche) et sortie complete du POC avec le patch L3 actif (droite).

E.3 Ce que cette demonstration etablit

Couche	Verdict	Preuve
Hooks user-mode SophosED	Contournee	Init complete sous hooks (annexes C et D)
Anti-exploit HMPA SysCallX64	Contournee	Heuristique fondee sur le RIP du syscall : l'appel direct du prologue l'execute dans le .text de ntdll. Le spoofing de la seule adresse de retour etait insuffisant (process tue, annexe D), l'appel direct passe.
Console / alertes	Aucune alerte	Aucune remontee dans la console Sophos Central pendant toute la sequence (initialisation contournee, syscall execute, fin du processus)
Callbacks noyau / ETW-TI	Seule barriere restante	Le syscall est reellement execute : la couche noyau l'observe. Rien n'a ete correle ni alerte cote console pendant le test.

Point de rigueur

Le process s'est termine avec le code 0xC0000005 apres l'affichage de "POC termine", reaction post-execution non qualifiee (aucun evenement remontant dans la console). Elle n'affecte pas le resultat : le syscall etait execute et sa reponse traitee.

E.4 Recommandations actualisees

- **Priorite 1, detection noyau obligatoire** : les deux couches user-mode Sophos sont contournables par des modifications mineures d'un outil public. Detection par stack-walking dans les callbacks (reperer les appels de prologues ntdll directs depuis des modules tiers), ETW-TI remontee jusqu'au moteur, et correlation en alerte visible : le test montre que la telemetrie existante ne s'est transformee en aucune alerte.
- **Priorite 2, prevention materielle** : HVCI/VBS et l'integrite du flux de controle rendent la creation et l'execution de stubs prives nettement plus couteuses.
- **Priorite 3, regle comportementale** : alerte sur la sequence chargement de DLL, allocation RX privee, appel de prologue ntdll en dehors du chemin API normal. Signature observee chez ABE, generalisable.

Constat final

L'experience prouve que "EDR installe" ne vaut rien sans "telemetrie noyau exploitee et correlee en alertes". Les hooks et l'anti-exploit user-mode retardent un attaquant (plusieurs etudes, plusieurs patches, plusieurs heures), ils ne l'arretent pas.

Fin du rapport. Documents lies : snapshots VM Proxmox, exports de journaux (Observateur d'evenements) et DLL de test archivees dans le dossier du projet.

References

[1] Signal Labs. "In-Memory Disassembly for EDR/AV Unhooking" - analyse des hooks Sophos (exemple etudie : NtProtectVirtualMemory, hook vers hmpalert.dll) et localisation du stub syscall original relocalise par desassemblage en memoire. Avril 2023. https://signal-labs.com/analysis-of-edr-hooks-bypasses-amp-our-rust-sample/